

Project Report

FPGA-Based

Anti-Theft System for Vehicles

Endalew Basazine Belay 1093908

Kassahun Enyew Aweke 1093899

Yordanos Fentahun Zerihun 1093894

SUPERVISED BY: ENG. GASM EL HASSAN,
DR. MOHAMMAD AWAD



Submitted: January 6, 2026

Contents

1	Introduction	5
1.1	Background and Motivation	5
1.2	Problem Statement	5
1.3	Literature Review	5
2	Design	6
2.1	Requirements, Constraints, and Considerations	6
2.2	Design Process	7
2.3	System Overview	12
2.4	2Component Design	14
3	Experimental Testing and Results	16
3.1	Testing Plan and Acceptance Criteria	16
3.2	Results	19
3.3	Analysis and Interpretation of Data	22
4	Conclusion	23
4.1	Summary	23
4.2	Future Improvements and Takeaways	23
4.3	Lessons Learned	24
4.4	Team Dynamics	24

List of Figures

1	RTL View of the Car Door Lock System FSM in Quartus II .	13
2	System in <code>DIGIT_ENTRY</code> state during PIN entry (Display: Entry).	20
3	System in <code>UNLOCKED</code> state after correct PIN entry (Display: ULOC, <code>UNLOCK_LED</code> ON).	21
4	ALARM state triggered after 3 incorrect attempts (Display: ALAR, buzzer active).	21
5	System in <code>TIMEOUT</code> state during lockout (Display: LOCK, system unresponsive to input).	22

List of Tables

1	Simulation Result	10
2	FPGA Pin Assignment for Input and Output Signals	12
3	FSM State Transition Table	14

Abstract

The project aims to create, test and implement a digital car door lock system in VHDL and with FPGA technology. The main goal is to establish a system where users' access is controlled with a PIN code and get instant feedback. An FSM is responsible for monitoring and switching among locked, unlocked and alarm states of the system. To confirm that the design is correct, simulation on EDAPlayground is used to ensure that both functions and timing are correct. The software is then run on an Altera DE2 FPGA board with Quartus II. The status is conveyed to the user through LEDs, a buzzer and a 7-segment display. Once more than three incorrect PIN codes are entered, the system goes into an alarm state and needs a reset to function normally again. The accuracy and reliability of the system have been confirmed, showing that FPGA-based systems are effective for real-time digital security applications.

1 Introduction

1.1 Background and Motivation

Due to car theft being a serious issue worldwide, having strong vehicle security systems is now more important than ever. Simple mechanical locks do not provide much security, since they are easy to force open or copy. For this reason, electronic locking systems are increasing in use due to their flexibility and defenses against trying to pick them. It involves designing a safe car door locking system that depends on Field Programmable Gate Arrays (FPGAs) which allow for quick and reliable logic operations.

The purpose of this work is to use FPGAs' flexibility to design a practical and PIN-guided anti-theft scheme. While microcontrollers function sequentially, FPGAs work in parallel which means they respond more quickly to security issues. Thanks to the use of LEDs, a buzzer and a 7-segment display, you can tell who is trying to enter your house and stay aware of any potential risks.

1.2 Problem Statement

The purpose of this project is to build a car door locking system that can confirm a user's identity using a 4-digit PIN they enter. The system must keep user data safe, reject any wrong inputs and activate the alarm when three incorrect attempts are made. Also, the system should show its current condition, whether it is Locked, Unlocked or Alarm, using both 7-segment displays and LEDs. There is a reset feature that helps restore the entire system to how it was first launched.

1.3 Literature Review

FPGAs are now key in building embedded systems since they are fast, great at reconfiguration and useful for real-time data handling. The ability of FPGAs to run several logic blocks together means digital locks built with FPGAs are faster and more responsive than those built with microcontrollers [1]. The use of an FPGA in a study by Umbarkar et al. allowed the digital lock to react faster and be more scalable, making it perfect for vehicle applications instead of MCU-based versions [2].

Finite state machines (FSMs) are well-known in hardware-based access control systems. FSMs let you define steps that always follow the same path when PIN pads are accessed, making handling these states stable and safe. It was noted by Nasir et al. that FSMs are well suited for creating lock mechanisms when built with Verilog or VHDL, especially along with alarms and visual displays [3].

Sometimes, digital locks use 7-segment displays and buzzers to help users understand the lock's operation. They give instant status reports, making the system both more convenient and reliable. As stated by Grout in his book on digital systems, embedded systems that deal with human interaction depend on feedback components [4]. Such feedback is also crucial in car security systems so that drivers learn of the lock status or any security violation quickly.

All things considered, introducing PIN-based digital locking on an FPGA enables both safe access and the addition of more security features over time. Examples could be PIN codes of more digits, Bluetooth technology and fingerprint recognition. The fact that FPGAs are reprogrammable allows modern car anti-theft systems to benefit from new features without major changes to hardware.

2 Design

2.1 Requirements, Constraints, and Considerations

There are many things that a digital car door lock system should be able to do. It has to read in a 4-digit PIN from the user and check it against the stored PIN. The system has to deny access after three consecutive failed attempts and make a visual and audible alarm sound at that time. It has to offer a method to restore the system to its locked state. It should be possible to recognize all three conditions, Locked, Unlocked and Alarm, via the use of 7-segment displays and indicator LEDs.

The software must manage the shortage of hardware to receive inputs, must provide instant response and handle user input debouncing successfully. Be-

cause PINpad works with a 4-bit interface, only digital PINs are possible. The system has to avoid letting rapid or simultaneous inputs cause the system to start behaving unpredictably.

Edge detection should be used to handle keystrokes, there should be counters to keep track of how many times a password is entered, and the keyboard status information needs to be constant. The design of the finite state machine (FSM) should ensure smooth movement between states and VHDL modules should be upgradable to support longer PINs and extra input methods in the future.

2.2 Design Process

An iterative approach is used to create the digital car door lock system. The purpose is to put in place a secure, immediate PIN-based access control system for an FPGA platform. Stages of the design process involve writing requirements, creating VHDL models, simulating them, creating hardware samples and adjusting based on testing results.

Initial Planning and Approach

As the FPGA platform is already set, the main part of the design process is to determine the optimum control system and functionality for the system to meet its objectives. After analyzing what we want to achieve, we conclude that a finite state machine (FSM) is the best way to manage all the required actions, user input, and errors.

After reviewing scalability, user interaction, and visual feedback, we opt for the modular FSM approach since it makes the design more maintainable and extendable. The FSM has six main states: **IDLE**, **DIGIT_ENTRY**, **VERIFY**, **ALARM**, **UNLOCKED**, and **UNLOCK_DELAY**.

Steps:

Step 1: Initial FSM Architecture

The FSM design starts with the following states:

- **IDLE**: Represents the lock state.
- **DIGIT_ENTRY**: Collects a 4-digit PIN from the user.

- **VERIFY:** Compares the entered PIN to a stored correct PIN.
- **UNLOCKED:** Grants access if the entered PIN is correct.
- **ALARM:** Triggers after 3 failed attempts to enter the correct PIN.
- **UNLOCK_DELAY:** Adds a post-unlock delay before returning to the IDLE state.

DIGIT_ENTRY State:

The DIGIT_ENTRY state accepts password PINs via `Key_in` on valid key processing. When a key edge is detected and fewer than four digits have been entered, the digit is appended to `entered_pin`, and the digit counter is incremented.

```

1 when DIGIT_ENTRY =>
2   if key_edge = '1' then
3     if digit_count < 4 then
4       entered_pin <= entered_pin(11 downto 0) & Key_in;
5       digit_count <= digit_count + 1;
6     end if;
7   end if;

```

IDLE State:

The IDLE state represents the locked state. It resets the PIN entry and waits for user input. If the system is currently in a lockout condition (`timeout_active = '1'`), it decrements the lock timer until the timeout expires. Once expired, it resets the attempt counter and exits the lockout.

```

1 when IDLE =>
2   entered_pin <= (others => '0');
3   digit_count <= 0;
4   if timeout_active = '1' then
5     if lock_timer > 0 then
6       lock_timer <= lock_timer - 1;
7     else
8       timeout_active <= '0';
9       attempt_count <= 0;
10    end if;
11  end if;

```

VERIFY State

The VERIFY state checks whether the entered PIN matches the stored correct PIN. If the PIN is incorrect, the attempt counter is incremented. After the third incorrect attempt, the system activates a timeout and transitions to the ALARM state. If the PIN is correct, the attempt counter is reset and the system transitions to the UNLOCKED state.

```
1 when VERIFY =>
2   if entered_pin /= correct_pass then
3     if attempt_count < 3 then
4       attempt_count <= attempt_count + 1;
5     else
6       timeout_active <= '1';
7       lock_timer <= 1000000000;
8     end if;
9   else
10    attempt_count <= 0;
11    unlock_timer <= 1000000000;
```

ALARM State

The ALARM state is triggered after three failed attempts to enter the correct PIN. It activates both visual and audio indicators such as a buzzer and LEDs. After a set lock period, the system resets the timeout and returns to the IDLE state.

```
1 when ALARM =>
2   if lock_timer > 0 then
3     lock_timer <= lock_timer - 1;
4   else
5     timeout_active <= '0';
6     attempt_count <= 0;
```

UNLOCKED State:

The UNLOCKED state is activated when the user enters the correct PIN. It turns on the unlocked LED and keeps the system in the unlocked state for a limited duration using the `unlock_timer`. Once the timer expires, the system transitions to the next state.

```

1 when UNLOCKED =>
2   if unlock_timer > 0 then
3     unlock_timer <= unlock_timer - 1;
4   end if;

```

UNLOCK_DELAY State:

The UNLOCK_DELAY state introduces a delay buffer before transitioning the system back to the IDLE state. This ensures a smoother user experience and helps debounce state transitions.

```

1 when UNLOCK_DELAY =>
2   next_state <= IDLE;

```

Step 2: Simulation and Functional Testing

The simulation is conducted using EDAPlayground by providing the complete VHDL implementation and an appropriate testbench. The testbench simulates user input of a 4-digit PIN using valid key signals with delays.

Sample Testbench Code:

```

1 Key_valid <= '1'; Key_in <= "0001"; wait for 20 ns;
2 Key_valid <= '0'; wait for 40 ns;
3
4 Key_valid <= '1'; Key_in <= "0010"; wait for 20 ns;
5 Key_valid <= '0'; wait for 40 ns;
6
7 Key_valid <= '1'; Key_in <= "0011"; wait for 20 ns;
8 Key_valid <= '0'; wait for 40 ns;
9
10 Key_valid <= '1'; Key_in <= "0100"; wait for 20 ns;
11 Key_valid <= '0'; wait for 200 ns;

```

Table 1: Simulation Result

Scenario	Expected Behavior	Observed State
Correct PIN	Access granted	UNLOCKED
3 Incorrect Attempts	Alarm triggered	ALARM
Reset pressed	Return to initial state	IDLE

Step 3: FPGA Implementation Using Quartus II

After the design passes simulation, Intel Quartus II is used to implement the code for the Altera DE2 FPGA development board. The VHDL files are imported into a new Quartus project, and the target FPGA is specified accordingly. The main module (`projectTest`) is defined, and the **Synthesis**, **Fitter**, and **Assembler** tools are run to generate the design.

All input and output signals are allocated to physical pins on the DE2 board based on the appropriate hardware mapping. The configuration includes:

- **Switches:** Used for `Key_in`.
- **Pushbuttons:** Used for `RESET` and `Key_valid`.
- **LEDs, Buzzer, and 7-Segment Display:** Used as output indicators.

After completing pin assignments, the design is checked for:

- Syntax errors
- Resource conflicts
- Timing issues

Once the compilation is successful, the Quartus Programmer is used to upload the configuration file (`.sof`) to the FPGA. At this stage, the design is physically deployed onto the DE2 platform for direct testing using the onboard hardware components.

Table 2: FPGA Pin Assignment for Input and Output Signals

Signal	Function	FPGA Pin	Board Element
Input Signals			
CLK	50 MHz Clock	PIN_N2	Onboard Oscillator
RESET	System Reset	PIN_G26	KEY[0]
Key_in[3]	PIN digit MSB	PIN_AE14	SW[3]
Key_in[2]	PIN digit	PIN_P25	SW[2]
Key_in[1]	PIN digit	PIN_N26	SW[1]
Key_in[0]	PIN digit LSB	PIN_N25	SW[0]
Key_valid	Confirm button	PIN_N23	KEY[1]
Output - LED Indicators			
ALARM_LED	Alarm indicator	PIN_AE23	LEDR[0]
LOCK_LED	Locked state LED	PIN_AF23	LEDR[1]
UNLOCK_LED	Unlocked state LED	PIN_AB21	LEDR[2]
Output - Buzzer			
Buzzer	Buzzer Output	PIN_D25	GPIO_0[0]
Output - 7-Segment Display (HEX0)			
SEG_DATA[0]	Segment a	PIN_AF10	HEX0[0]
SEG_DATA[1]	Segment b	PIN_AB12	HEX0[1]
SEG_DATA[2]	Segment c	PIN_AC12	HEX0[2]
SEG_DATA[3]	Segment d	PIN_AD11	HEX0[3]
SEG_DATA[4]	Segment e	PIN_AE11	HEX0[4]
SEG_DATA[5]	Segment f	PIN_V14	HEX0[5]
SEG_DATA[6]	Segment g	PIN_V13	HEX0[6]

2.3 System Overview

The following section describes the overall structure of the car door lock system working on FPGA hardware. The Register Transfer Level (RTL) view is employed to illustrate the structure that results from VHDL code synthesis. It displays how internal signals, components and modules work together on a structural level in the FPGA.

RTL View:

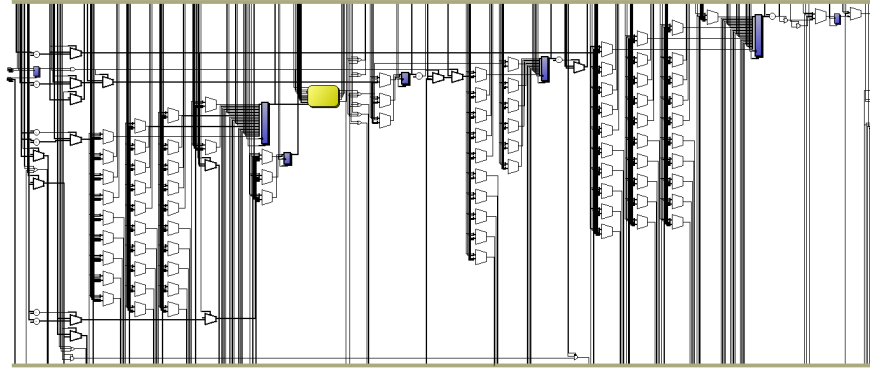


Figure 1: RTL View of the Car Door Lock System FSM in Quartus II

System Description

The design has four main functional blocks arranged in a modular way.

Input Interface: The system uses SW[3:0] to enter 4-bit digit values, and KEY[1] to confirm each digit. The Key_valid signal is produced from this input to detect valid shifts.

FSM Controller: This is an essential part of the system. Based on user input, the system's previous state, the number of tries, and timeouts, the FSM decides the system's next behavior. It transitions to IDLE when inactive, DIGIT_ENTRY when entering digits, VERIFY when a pin code entry completes, ALARM for security breaches, and UNLOCKED after successful authentication.

Output Logic: This block controls system indicators, including LOCK_LED, UNLOCK_LED, and ALARM_LED. It triggers the buzzer during the ALARM mode and resets all outputs upon returning to IDLE.

Display Driver: A dedicated logic unit cycles through the designated digits on the 7-segment display. Status messages such as "LOCK", "ULOC", and "ALAR" are shown using a multiplexer counter and segment encodings.

The RTL view illustrates how these components interconnect via signals like `Key_in`, `Key_valid`, `state`, and various control outputs. This verifies that synthesis is consistent with the VHDL source code.

2.4 2Component Design

The primary components include the Finite State Machine (FSM), Edge Detector, Attempt Counter, 7-Segment Display Multiplexer, and Buzzer Logic.

Finite State Machine (FSM)

The FSM is the central part of the whole system. It runs the lock logic by following strictly defined state changes. The FSM includes six states: `IDLE`, `DIGIT_ENTRY`, `VERIFY`, `ALARM`, `UNLOCKED`, and `UNLOCK_DELAY`.

Table 3: FSM State Transition Table

Current State	Input Condition	Next State
IDLE	Valid key press (<code>Key_valid</code>)	DIGIT_ENTRY
DIGIT_ENTRY	4 digits entered	VERIFY
VERIFY	PIN correct	UNLOCKED
VERIFY	PIN incorrect (<3 attempts)	IDLE
VERIFY	PIN incorrect (3rd attempt)	ALARM
ALARM	Timeout expired	IDLE
UNLOCKED	Unlock delay expired	UNLOCK_DELAY
UNLOCK_DELAY	Always	IDLE

Key Press Edge Detection

Edge detection in the system prevents multiple detections of a single switch press. This is implemented as:

```
1 key_edge <= Key_valid and not key_pressed_prev;
```

Attempt Counter Logic

A counter keeps track of the number of unsuccessful PIN attempts. When the counter reaches 3, the system activates the alarm and buzzer, then initiates a timeout using the `lock_timer`.

```

1 if entered_pin /= correct_pass then
2   if attempt_count < 3 then
3     attempt_count <= attempt_count + 1;
4   else
5     timeout_active <= '1';
6     lock_timer <= timeout_limit;
7   end if;
8 end if;

```

Timer/Delay Counters

The system uses counters for both of the following purposes:

- `lock_timer` – Activates the `ALARM` state to manage a timeout period.
- `unlock_timer` – Active only when the system is unlocked, allowing temporary access.

Every clock cycle, the corresponding counter decreases until it expires, triggering the next state transition.

Display Multiplexer

The system displays messages such as “LOCK” and “UNLOCK” on a 4-digit 7-segment display by multiplexing the digits in timed intervals.

Mux Logic Pseudocode:

```

1 on rising_edge(CLK)
2   case seg_mux_counter is
3     when 0 =>
4       SEG_SELECT <= "1110";
5       SEG_DATA <= display_string(6 downto 0);
6     when 1 =>
7       SEG_SELECT <= "1101";
8       SEG_DATA <= display_string(13 downto 7);
9     when 2 =>
10        SEG_SELECT <= "1011";
11        SEG_DATA <= display_string(20 downto 14);
12    when 3 =>

```

```

13     SEG_SELECT <= "0111";
14     SEG_DATA <= display_string(27 downto 21);
15 end case;
16 seg_mux_counter <= (seg_mux_counter + 1) mod 4;

```

Buzzer Activation Logic

The buzzer is activated exclusively when the system is in the ALARM state.

```

1 if state = ALARM then
2     Buzzer <= '1';
3 else
4     Buzzer <= '0';
5 end if;

```

3 Experimental Testing and Results

3.1 Testing Plan and Acceptance Criteria

Ongoing tests are made with different inputs to check if the system handles all the possible ways a user can enter a PIN, triggers the alarm, resets properly, and updates the screen content. After Quartus II successfully deploys the hardware, the test process is performed on the FPGA.

Test case 1: Correct PIN entry

- **Test name:** Valid Unlock
- **Test description:** This indicates that the system enters the UNLOCKED state after all four digits (1234) of the PIN are entered one after the other.
- **Steps:**
 - Press RESET to start the DIGIT_ENTRY state.
 - Enter the correct digits: 0001, 0010, 0011, 0100 via SW[3:0].
 - After entering each digit, press KEY[1] to confirm and store the input.

- **Expected Results:**
 - Unlock LED ON.
 - 7-segment display shows U.
 - Buzzer and Alarm are OFF.
- **Observed Result:**
 - The system successfully entered UNLOCKED state.
 - Unlock LED is ON and U is shown on the segment.
- **Acceptance Criteria:** A correct PIN is required for unlock; the display updates and LED work normally.
- **Test Result:** Pass.

Test case 2: Single Wrong PIN entry Attempt

- **Test name:** First Failed Attempt
- **Test description:** Matches the system's response after 1 incorrect PIN is entered.
- **Steps:**
 - Press RESET to start the DIGIT_ENTRY state.
 - Enter the digits 0001, 0110, 0101, 0000 using SW[3:0].
 - After entering each digit, press KEY[1] to confirm and store the input.
- **Expected Results:**
 - System turns into the IDLE state.
 - Locked LED remains ON.
 - Buzzer and Alarm are OFF.
- **Observed Result:**
 - The system entered the locked state.
 - Lock LED is ON, and U is shown on the segment.

- **Acceptance Criteria:** A wrong PIN cannot trigger unlock or alarm.
- **Test Result:** Pass.

Test case 3: Alarm Activation

- **Test name:** Alarm after 3 incorrect PIN attempts
- **Test description:** When a user takes 3 wrong tries in a row, the system switches to the **ALARM** state.
- **Steps:**
 1. Enter 3 wrong PINs consecutively.
 2. Observe display, LED, and buzzer.
- **Expected Results:**
 - System turns into the **ALARM** state.
 - Alarm LED and Buzzer turn ON.
 - 7-segment display shows **A**.
- **Observed Result:**
 - Alarm is triggered right after 3 incorrect attempts.
 - LED and Buzzer active.
 - The segment displays **A**.
- **Acceptance Criteria:** Alarm will be activated after 3 failed attempts.
- **Test Result:** Pass.

Test case 4: Unlock Delay Expiry

- **Test name:** Unlock Timeout
- **Test description:** Ensures that the system switches to **IDLE** after the time in the **UNLOCKED** state runs out.
- **Steps:**

1. Enter the right PIN.
2. Wait for some seconds.

- **Expected Results:**

- System turns into the IDLE state.
- U changes to L on the 7-segment display.
- Unlock LED turns OFF, and Lock LED turns ON.

- **Observed Result:**

- Delay expired, system returned to IDLE state.
- LOCK.LED turned ON.
- Display changed as expected.

- **Acceptance Criteria:** After the unlock timeout, the system enters the UNLOCK_DELAY state for UNLOCK_DURATION, and then it must return to the IDLE state.

- **Test Result:** Pass

3.2 Results

The car door lock system designed with FPGA operates consistently and produces accurate outputs under all test conditions. The design was implemented on the Altera DE2 development board, and its operation was verified by observing the LEDs, the 7-segment display outputs, and the buzzer sound. The finite state machine (FSM) transitions states based on user inputs, and the interface provides clear visual and audio feedback confirming the current state.

When a user enters their 4-digit PIN, the system transitions into the DIGIT_ENTRY state. At this point, the system accepts one digit at a time from SW[3:0], and each digit entry is confirmed by pressing KEY[1]. While digits are being entered, the system displays LOCK on the 7-segment display, and the LOCK.LED remains illuminated, as shown in figure 2. The edge detection logic ensures that only intentional key presses are registered, preventing duplicate inputs caused by switch bouncing.

After all four digits have been entered, the FSM moves to the **VERIFY** state. If the entered PIN matches the stored password, the system unlocks and enters the **UNLOCKED** state. In this state, the display shows **ULOC**, and the **UNLOCK_LED** is lit, signaling that access has been granted. The system remains in this state briefly, governed by the unlock timer, before transitioning to an idle state.

If an incorrect PIN is entered, the system increments the attempt counter internally. Upon three consecutive failed attempts, the FSM switches to the **ALARM** state. At this point, the buzzer sounds, and the **ALARM_LED** is activated. The 7-segment display shows **ALAR** immediately to notify the user, as depicted in Figure 3. All other functions are disabled until the **lock_timer** expires, preventing further interaction.

Following the alarm period, the FSM blocks all system activity for the full lockout duration. This timeout period, illustrated in Figure 4, enhances security by preventing rapid repeated attempts, effectively protecting the system from brute-force attacks. Once the lock timer expires, the FSM returns to the **IDLE** state, clears the previous attempts, and is ready to accept new input.

The consistent behavior of the hardware across all test scenarios confirms the correctness of the FSM implementation. All LEDs, the buzzer, and the 7-segment display produce the expected outputs in each state, and no timing or functional issues were observed during testing.

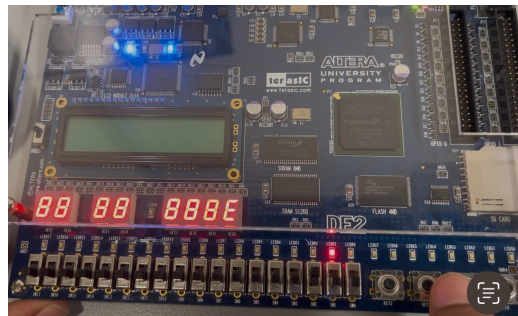


Figure 2: System in **DIGIT_ENTRY** state during PIN entry (Display: Entry).

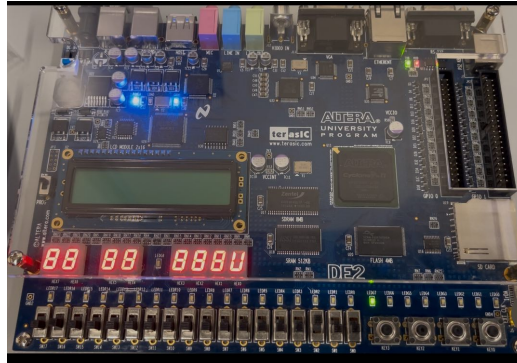


Figure 3: System in UNLOCKED state after correct PIN entry (Display: ULOC, UNLOCK_LED ON).

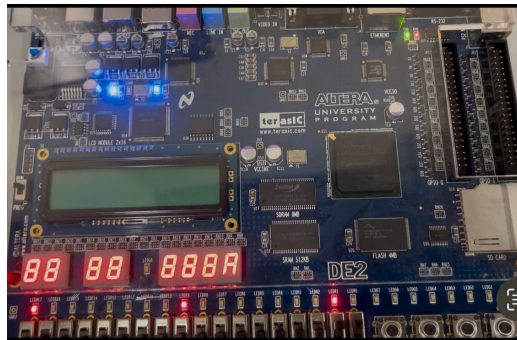


Figure 4: ALARM state triggered after 3 incorrect attempts (Display: ALAR, buzzer active).

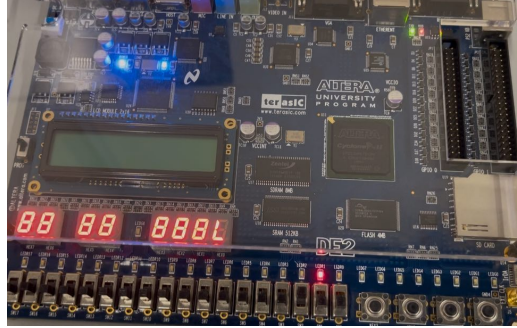


Figure 5: System in TIMEOUT state during lockout (Display: LOCK, system unresponsive to input).

3.3 Analysis and Interpretation of Data

Experiments prove that the system functions properly, as each state transition occurs when the proper user input is given. The accurate way the system enters data and identifies the correct PIN confirms that these parts of the code work correctly. With edge detection, there is no chance of the keyboard triggering the wrong input due to switch bounce.

After three incorrect attempts, the system transitions to the **ALARM** mode and enforces a timed lockout, demonstrating that security has been effectively achieved. The screen, LEDs, and buzzer provide clear indicators of a security breach. The consistent behavior of these outputs supports that the FSM operates as expected under real conditions.

The software runs smoothly on hardware as anticipated, indicating that the design is correct and free of errors. However, there is some slight flicker when the display reaches its maximum refresh rate, which could be improved in future designs. Similarly, the timers rely on counting clock cycles, which is suitable for short durations but may require more precise timing for extended applications.

Overall, the testing and results show that the design accomplishes its intended functions. Every system action is predictable, and no unexpected behavior was observed during testing of different scenarios. While some improvements to the user interface could enhance performance, the system is robust and meets the requirements for secure access control.

4 Conclusion

4.1 Summary

This project achieves a secure way to open and close car doors using VHDL and an FPGA board. The system handles PIN entry, verifies the PIN, activates alarms, and automatically resets when necessary by employing a finite state machine (FSM) along with audio and visual signals. Through both simulation and hardware testing, the design demonstrates accurate PIN handling, reliable alarm triggering upon multiple failed attempts, proper timeout functionality, and clear state transitions.

The results confirm that the FSM behaves correctly in simulated and practical environments, validating the logic and design methodology. This project illustrates how FPGAs can be used to build secure embedded systems that operate in real-time and allow for future reconfiguration. Applying theoretical concepts in a practical setting enhanced our understanding of digital systems, FSMs, and VHDL programming.

4.2 Future Improvements and Takeaways

While the system performs its intended function well, several enhancements could improve its features and user-friendliness:

- Extending the PIN length beyond four digits would enhance security by increasing the number of possible combinations.
- Integrating biometric authentication, such as fingerprint scanning, either alongside or instead of PIN entry could reduce input errors and provide a smoother login experience.
- Adding Bluetooth connectivity would enable users to lock and unlock their car doors remotely via a smartphone app. This would require interfacing a wireless communication module with the FPGA and developing secure communication protocols.

The project demonstrates that FPGA-based solutions are well-suited for security applications and offer flexibility for future expansion. Adding new features to the FSM design requires minimal effort, showcasing the power and adaptability of hardware design in embedded systems.

4.3 Lessons Learned

This project provided extensive practical experience with VHDL, testbench creation, and FSM development. Many team members had no prior experience using EDAPlayground for simulation or Quartus II for FPGA implementation.

Key skills acquired include:

- Writing and debugging VHDL code for hardware logic.
- Using simulation tools to verify design correctness.
- Assigning FPGA pins and managing designs in Quartus II.
- Applying theoretical FSM models in practical embedded system projects.

We also learned the importance of modular testing, precise pin assignments, and early simulation to catch hardware layout errors. Tackling edge detection and refining state-saving logic were crucial lessons that improved the final design.

4.4 Team Dynamics

Our team consisted of three members, with Kassahun serving as the leader. Leadership responsibilities included managing work schedules, monitoring development milestones, and maintaining adherence to the project timeline. Communication was facilitated via a WhatsApp group, and the team regularly met in the university lab and library to discuss progress and resolve challenges.

Endalew was responsible for VHDL programming, while Yordanos handled simulation and documentation tasks. Regular distribution of meeting minutes and progress updates ensured smooth coordination. Our primary goal was to migrate the simulation model onto the physical FPGA, which we successfully achieved. Every team member contributed meaningfully, and our collaborative effort enabled effective problem-solving and project completion satisfaction.

Appendix A: VHDL Source Code

A.1 Main FSM Module — projectTest.vhd

Listing 1: Main VHDL Implementation: projectTest.vhd

```
1
2 library IEEE;
3
4 use IEEE.NUMERIC_STD.ALL;
5
6 use IEEE.STD_LOGIC_1164.ALL;
7
8 entity projectTest is
9
10     Port(
11
12         CLK : in std_logic;
13
14         RESET : in std_logic;
15
16         Key_in : in std_logic_vector(3 downto 0);
17
18         Key_valid : in std_logic;
19
20         ALARM_LED : out std_logic;
21
22         STATUS_7segmentDisplay : out std_logic_vector(6 downto 0);
23
24         Buzzer : out std_logic;
25
26         LOCK_LED : out std_logic;
27
28         UNLOCK_LED : out std_logic;
29
30         SEG_SELECT : out std_logic_vector(3 downto 0);
31
32         SEG_DATA : out std_logic_vector(6 downto 0)
33
34     );
```

```

35
36 end projectTest;
37
38 architecture Behavioral of projectTest is
39
40     type State_Type is (IDLE, DIGIT_ENTRY, VERIFY, ALARM, UNLOCKED,
41                         UNLOCK_DELAY);
42
43     signal state, next_state : State_Type;
44
45     signal correct_pass : std_logic_vector(15 downto 0) :=
46         "0001001000110100"; -- PIN: 1234
47
48     signal entered_pin : std_logic_vector(15 downto 0) := (others =>
49         '0');
50
51     signal digit_count : integer range 0 to 4 := 0;
52
53     signal attempt_count : integer range 0 to 4 := 0;
54
55     signal lock_timer : integer range 0 to 2000000000 := 0;
56
57     signal unlock_timer : integer range 0 to 2000000000 := 0;
58
59     signal timeout_active : std_logic := '0';
60
61     signal key_pressed_prev : std_logic := '0';
62
63     signal key_edge : std_logic := '0';
64
65     signal display_string : std_logic_vector(27 downto 0);
66
67     signal seg_mux_counter : integer range 0 to 3 := 0;
68
69     signal SEG_DATA_INTERNAL : std_logic_vector(6 downto 0);
70
71     constant SEG_L : std_logic_vector(6 downto 0) := "1000111";
72
73     constant SEG_0 : std_logic_vector(6 downto 0) := "1000000";

```

```

72     constant SEG_C : std_logic_vector(6 downto 0) := "1000110";
73
74     constant SEG_U : std_logic_vector(6 downto 0) := "1000001";
75
76     constant SEG_A : std_logic_vector(6 downto 0) := "0001000";
77
78     constant SEG_DASH : std_logic_vector(6 downto 0) := "0000001";
79
80 begin
81
82     key_edge <= Key_valid and not key_pressed_prev;
83
84     process(CLK, RESET)
85
86     begin
87
88         if RESET = '0' then
89
90             state <= IDLE;
91
92             entered_pin <= (others => '0');
93
94             digit_count <= 0;
95
96             attempt_count <= 0;
97
98             lock_timer <= 0;
99
100            unlock_timer <= 0;
101
102            timeout_active <= '0';
103
104            key_pressed_prev <= '0';
105
106        elsif rising_edge(CLK) then
107
108            state <= next_state;
109
110            key_pressed_prev <= Key_valid;
111

```

```

112     case state is
113
114         when IDLE =>
115
116             entered_pin <= (others => '0');
117
118             digit_count <= 0;
119
120             if timeout_active = '1' then
121
122                 if lock_timer > 0 then
123
124                     lock_timer <= lock_timer - 1;
125
126                 else
127
128                     timeout_active <= '0';
129
130                     attempt_count <= 0;
131
132                 end if;
133
134             end if;
135
136         when DIGIT_ENTRY =>
137
138             if key_edge = '1' then
139
140                 if digit_count < 4 then
141
142                     entered_pin <= entered_pin(11 downto 0) & Key_in;
143
144                     digit_count <= digit_count + 1;
145
146                 end if;
147
148             end if;
149
150         when VERIFY =>
151

```

```

152         if entered_pin /= correct_pass then
153
154             if attempt_count < 3 then
155
156                 attempt_count <= attempt_count + 1;
157
158             else
159
160                 timeout_active <= '1';
161
162                 lock_timer <= 1000000000;
163
164             end if;
165
166         else
167
168             attempt_count <= 0;
169
170             unlock_timer <= 1000000000;
171
172         end if;
173
174     when ALARM =>
175
176         if lock_timer > 0 then
177
178             lock_timer <= lock_timer - 1;
179
180         else
181
182             timeout_active <= '0';
183
184             attempt_count <= 0;
185
186         end if;
187
188     when UNLOCKED =>
189
190         if unlock_timer > 0 then
191

```

```

192         unlock_timer <= unlock_timer - 1;
193
194     end if;
195
196     when others => null;
197
198 end case;
199
200 end if;
201
202 end process;
203
204 process(state, key_edge, digit_count, entered_pin, correct_pass,
        attempt_count, lock_timer, timeout_active, unlock_timer)
205
206 begin
207
208     next_state <= state;
209
210     case state is
211
212     when IDLE =>
213
214         if timeout_active = '0' and key_edge = '1' then
215
216             next_state <= DIGIT_ENTRY;
217
218             elsif timeout_active = '1' and lock_timer > 0 then
219
220                 next_state <= ALARM;
221
222             end if;
223
224     when DIGIT_ENTRY =>
225
226         if digit_count >= 4 then
227
228             next_state <= VERIFY;
229
230         end if;

```

```

231
232     when VERIFY =>
233
234         if entered_pin = correct_pass then
235
236             next_state <= UNLOCKED;
237
238         elsif attempt_count >= 3 then
239
240             next_state <= ALARM;
241
242         else
243
244             next_state <= IDLE;
245
246         end if;
247
248     when ALARM =>
249
250         if lock_timer = 0 then
251
252             next_state <= IDLE;
253
254         end if;
255
256     when UNLOCKED =>
257
258         if unlock_timer = 0 then
259
260             next_state <= UNLOCK_DELAY;
261
262         end if;
263
264     when UNLOCK_DELAY =>
265
266         next_state <= IDLE;
267
268     when others =>
269
270         next_state <= IDLE;

```

```

271     end case;
272
273
274 end process;
275
276 process(state, timeout_active)
277
278 begin
279
280     ALARM_LED <= '0';
281
282     Buzzer <= '0';
283
284     LOCK_LED <= '0';
285
286     UNLOCK_LED <= '0';
287
288     display_string <= SEG_DASH & SEG_DASH & SEG_DASH & SEG_DASH;
289
290     case state is
291
292         when IDLE =>
293
294             display_string <= SEG_L & SEG_0 & SEG_C & SEG_DASH;
295
296             LOCK_LED <= '1';
297
298             if timeout_active = '1' then
299
300                 ALARM_LED <= '1';
301
302             end if;
303
304         when DIGIT_ENTRY =>
305
306             display_string <= SEG_L & SEG_0 & SEG_C & SEG_DASH;
307
308             LOCK_LED <= '1';
309
310         when VERIFY =>

```

```

311         display_string <= SEG_DASH & SEG_DASH & SEG_DASH & SEG_DASH;
312
313         LOCK_LED <= '1';
314
315     when ALARM =>
316
317         display_string <= SEG_A & SEG_L & SEG_A & SEG_DASH;
318
319         ALARM_LED <= '1';
320
321         Buzzer <= '1';
322
323         LOCK_LED <= '1';
324
325     when UNLOCKED =>
326
327         display_string <= SEG_U & SEG_L & SEG_O & SEG_C;
328
329         UNLOCK_LED <= '1';
330
331     when UNLOCK_DELAY =>
332
333         display_string <= SEG_L & SEG_O & SEG_C & SEG_DASH;
334
335         LOCK_LED <= '1';
336
337     when others =>
338
339         display_string <= SEG_DASH & SEG_DASH & SEG_DASH & SEG_DASH;
340
341         LOCK_LED <= '1';
342
343     end case;
344
345 end process;
346
347 process(CLK)
348
349 begin
350

```

```

351
352     if rising_edge(CLK) then
353
354         case seg_mux_counter is
355
356             when 0 =>
357
358                 SEG_SELECT <= "1110";
359
360                 SEG_DATA_INTERNAL <= display_string(6 downto 0);
361
362             when 1 =>
363
364                 SEG_SELECT <= "1101";
365
366                 SEG_DATA_INTERNAL <= display_string(13 downto 7);
367
368             when 2 =>
369
370                 SEG_SELECT <= "1011";
371
372                 SEG_DATA_INTERNAL <= display_string(20 downto 14);
373
374             when others =>
375
376                 SEG_SELECT <= "0111";
377
378                 SEG_DATA_INTERNAL <= display_string(27 downto 21);
379
380         end case;
381
382         seg_mux_counter <= (seg_mux_counter + 1) mod 4;
383
384     end if;
385
386 end process;
387
388 SEG_DATA <= SEG_DATA_INTERNAL;
389
390 STATUS_7segmentDisplay <= SEG_DATA_INTERNAL;

```

```
391  
392 end Behavioral;
```

A.2 Testbench — `tb_projectTest.vhd`

Listing 2: VHDL Testbench: *tb_projectTest.vhd*

```
1  
2 library IEEE;  
3  
4 use IEEE.STD_LOGIC_1164.ALL;  
5  
6 use IEEE.NUMERIC_STD.ALL;  
7  
8 entity tb_projectTest is  
9  
10 end tb_projectTest;  
11  
12 architecture sim of tb_projectTest is  
13  
14     signal CLK : std_logic := '0';  
15  
16     signal RESET : std_logic := '1';  
17  
18     signal Key_in : std_logic_vector(3 downto 0) := (others => '0');  
19  
20     signal Key_valid : std_logic := '0';  
21  
22     signal ALARM_LED, Buzzer, LOCK_LED, UNLOCK_LED : std_logic;  
23  
24     signal STATUS_7segmentDisplay : std_logic_vector(6 downto 0);  
25  
26     signal SEG_SELECT : std_logic_vector(3 downto 0);  
27  
28     signal SEG_DATA : std_logic_vector(6 downto 0);  
29  
30     component projectTest  
31
```

```

32 Port (
33
34     CLK : in std_logic;
35
36     RESET : in std_logic;
37
38     Key_in : in std_logic_vector(3 downto 0);
39
40     Key_valid : in std_logic;
41
42     ALARM_LED : out std_logic;
43
44     STATUS_7segmentDisplay : out std_logic_vector(6 downto 0);
45
46     Buzzer : out std_logic;
47
48     LOCK_LED : out std_logic;
49
50     UNLOCK_LED : out std_logic;
51
52     SEG_SELECT : out std_logic_vector(3 downto 0);
53
54     SEG_DATA : out std_logic_vector(6 downto 0)
55
56 );
57
58 end component;
59
60 begin
61
62     UUT: projectTest port map (
63
64         CLK => CLK,
65
66         RESET => RESET,
67
68         Key_in => Key_in,
69
70         Key_valid => Key_valid,
71

```

```

72     ALARM_LED => ALARM_LED,
73
74     STATUS_7segmentDisplay => STATUS_7segmentDisplay,
75
76     Buzzer => Buzzer,
77
78     LOCK_LED => LOCK_LED,
79
80     UNLOCK_LED => UNLOCK_LED,
81
82     SEG_SELECT => SEG_SELECT,
83
84     SEG_DATA => SEG_DATA
85
86 );
87
88 -- Clock generation
89
90 CLK <= not CLK after 10 ns;
91
92 -- Stimulus process
93
94 process
95
96 begin
97
98     -- Reset the system
99
100    RESET <= '0';
101
102    wait for 20 ns;
103
104    RESET <= '1';
105
106    -- Enter correct PIN: 1 2 3 4 (binary: 0001 0010 0011 0100)
107
108    wait for 40 ns;
109
110    Key_in <= "0001"; Key_valid <= '1'; wait for 20 ns;
111

```

```

112     Key_valid <= '0'; wait for 40 ns;
113
114     Key_in <= "0010"; Key_valid <= '1'; wait for 20 ns;
115
116     Key_valid <= '0'; wait for 40 ns;
117
118     Key_in <= "0011"; Key_valid <= '1'; wait for 20 ns;
119
120     Key_valid <= '0'; wait for 40 ns;
121
122     Key_in <= "0100"; Key_valid <= '1'; wait for 20 ns;
123
124     Key_valid <= '0';
125
126     wait for 500 ns;
127
128     wait;
129
130 end process;
131
132 end sim;

```

References

- [1] I. Grout, *Digital Systems Design with FPGAs and CPLDs*. Oxford, U.K.: Elsevier, 2011.
- [2] S. Umbarkar, P. B. Bhoi, and A. R. Gawande, "FPGA-based digital door lock system," *Int. J. Adv. Res. Innov. Ideas Educ.*, vol. 9, no. 3, 2023.
- [3] N. M. Nasir, A. R. Kassim, M. H. Hassan, and N. A. A. Majid, "Electronic combination lock system using Verilog HDL," *Int. J. Adv. Technol. Eng. Explor.*, vol. 8, no. 75, pp. 328–335, 2021.
- [4] A. Kumar, "Real-time embedded system design using FPGA," *Int. J. Electron. Commun. Technol.*, vol. 6, no. 4, pp. 85–89, 2020.